Do NLP Models Know Numbers?

Probing Numeracy in Embeddings - EMNLP 2019

XIN SUN

Heidelberg University
Institute of Computer Science
Database Systems Research Group
ethan.sun921107@gmail.com

June 18, 2020

Outline

1. Introduction
2. NAQANet's numeracy on DROP
3. Probing tasks
4. Result Analysis

Outline

1. Introduction
2. NAQANet's numeracy on DROP
3. Probing tasks
4. Result Analysis

Numerical Reasoning

- Inspiration

  Most NLP models embed all tokens as distributed vectors. Is this enough to capture numeracy?

- Target

  - To investigate if and how "out-of-box" NLP models already learn numerical reasoning capabilities on DROP dataset.
  - To compare and analyze performance of numeracy on different embedding methods.

Concrete Steps

- Look at numeracy in NAQANet
- Stress testing NAQANet's numeracy
- How does numeracy appear with probing tasks
- Probing tasks on different embedding techniques

Outline

1. Introduction
2. NAQANet's numeracy on DROP
3. Probing tasks
4. Result Analysis

DROP dataset

DROP is a reading comprehension dataset that tests numerical reasoning operations such as counting, sorting, and addition, etc.

...JaMarcus Russell completed a 91-yard touchdown pass to rookie wide receiver **Chaz Schilens**. The Texans would respond with fullback Vonta Leach getting a 1-yard touchdown run, yet the Raiders would answer with kicker Sebastian Janikowski getting a 33-yard and a 21-yard field goal. Houston would tie the game in the second quarter with kicker Kris Brown getting a **53**-yard and a 24-yard field goal. Oakland would take the lead in the third quarter with wide receiver Johnnie Lee Higgins catching a **29-yard** touchdown pass from Russell, followed up by an 80-yard punt return for a touchdown.

Q: How many yards was the longest field goal? A: 53

Q: How long was the shortest touchdown pass? A: 29-yard

Q: Who caught the longest touchdown? A: Chaz Schilens

Figure: DROP dataset. (Eric et al 2019. Figure 2)

NAQANet

NAQANet follows typical reading comprehension model QANet: embedding, encoding, passage-question attention and output layers.

- Inputs
  - question representation.
  - a projected question-aware passage representation.
- Outputs
  - Passage span, Question span, Count, Arithmetic expression.
  - Answer type prediction.
- Embedding layer
  - GloVe
  - Char-CNN

Comparative and Superlative Questions

- Comparative questions

  Quantities or events that are "larger", "smaller", or "longer" than others.

- Superlative questions

  The "shortest", "largest", or "biggest" quantity in a passage.

Question Type	Example	Reasoning Required
Comparative (Binary)	Which country is a bigger exporter, Brazil or Uruguay?	Binary Comparison
Comparative (Non-binary)	Which player had a touchdown longer than 20 yards?	Greater Than
Superlative (Number)	How many yards was the shortest field goal?	List Minimum
Superlative (Span)	Who kicked the longest field goal?	Argmax

Figure: Comparative and Superlative questions.(Eric et al 2019. Table 1)

Comparative and Superlative Questions / 2

Question Type	Count	EM	F1
Human (Test Set)	9622	92.4	96.0
Full Validation	9536	46.2	49.2
Number Answers	5842	44.3	44.4
Comparative	704	73.6	76.4
Binary (either-or)	477	86.0	89.0
Non-binary	227	47.6	49.8
Superlative Questions	861	64.6	67.7
Number Answers	475	68.8	69.2
Span Answers	380	59.7	66.3

Figure: Emergent Numeracy in NAQANet.(Eric et al 2019. Table 2)

Stress Testing NAQANet's Numeracy

- Stress testing on two phenomena:
  - larger numbers.

    Generate a random positive integer and multiply or add that value to the numbers in each paragraph
  - word-form numbers.

    Replace every digit in the paragraph with its word form (e.g., "75" "seventy- five"). Word replacements for integers in the range [0, 100]

Stress Testing NAQANet's Numeracy / 2

Stress Test Dataset	All Questions F1	All Questions Δ	Superlative F1	Superlative Δ
Original Validation Set	49.2	-	67.7	-
Add [1, 20]	47.7	-1.5	64.1	-3.6
Add [21, 100]	41.4	-7.8	40.4	-27.3
Multiply [2, 10]	41.1	-8.1	39.3	-28.4
Multiply [11, 100]	38.8	-10.4	32.0	-35.7
Digits to Words [0, 20]	45.5	-3.7	63.8	-3.9
Digits to Words [21, 100]	41.9	-7.3	46.1	-21.6

Figure: Add or Multiply [x, y]. Or digits → words within range [x, y].(Eric et al 2019. Table 3)

Outline

1. Introduction
2. NAQANet's numeracy on DROP
3. Probing tasks
4. Result Analysis

Probing tasks

- List Maximum

  Given a list of embeddings for five numbers, the task is to predict the index of maximum number.

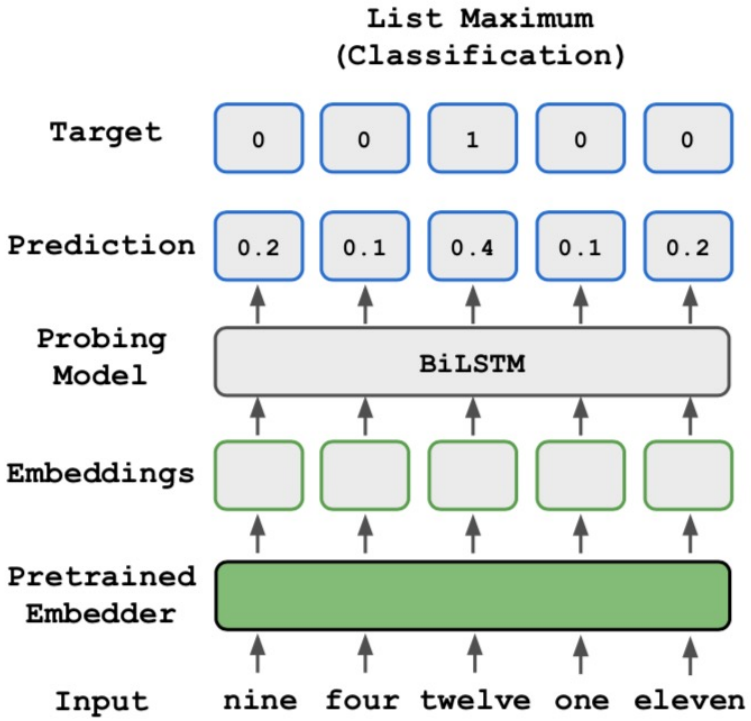

Figure: (Eric et al 2019. Figure 3)

Probing tasks / 2

- Number Decoding

  Given a number's embedding, the task is to regress to its value: the embedding for the string "five" has a target 5.0.

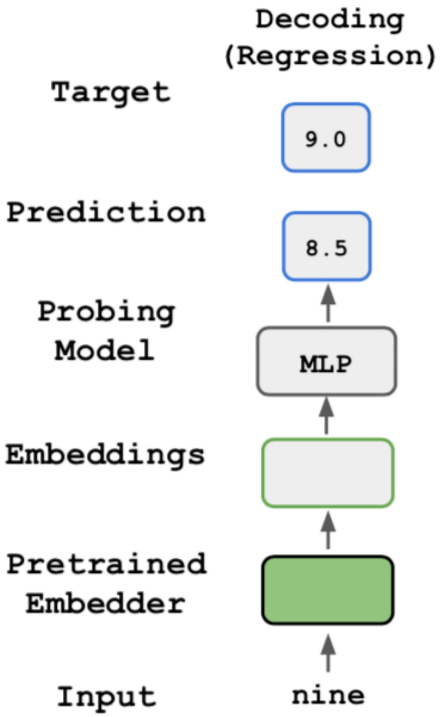

Figure: (Eric et al 2019. Figure 3)

Probing tasks / 3

- Addition task

  The addition task requires number manipulation — given the embeddings of two numbers, the task is to predict their sum.

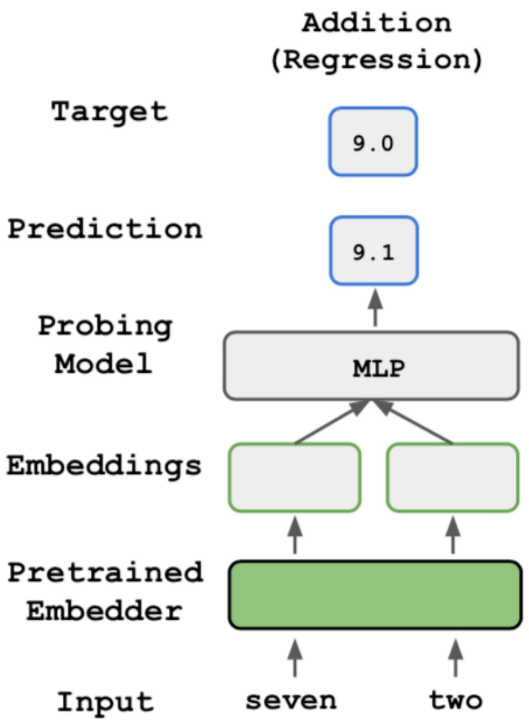

Figure: (Eric et al 2019. Figure 3)

Embedding Methods used in Probing tasks

The probing models are trained on top of following embeddings.

- Word Vectors - GloVe and word2vec

  300-dimensional GloVe and word2vec vectors.

- Contextualized Embeddings - ELMo and Bert

  ELMo uses Char-CNNs of size 1–7 with max pooling. BERT represents tokens via sub-word pieces; Lower-cased BERT-base with 30k pieces.

Embedding Methods used in Probing tasks / 2

- NAQANet Embeddings

  Extract GloVe embeddings and Char-CNN from NAQANet model trained on DROP.

- Learned Embeddings

  Use Char-CNN and Char-LSTM and left character padding.

- Number's Value as Embedding (Value Embedding)

  Simply map a number's embedding directly to its value (e.g., "seventy-five" embeds to [75]).

Embedding Methods used in Probing tasks / 3

- Untrained Embeddings - Two untrained baselines.
  - Random token vectors, which trivially fail to generalize
  - Randomly initialized and untrained Char-CNN and Char-LSTM

Outline

1. Introduction
2. NAQANet's numeracy on DROP
3. Probing tasks
4. Result Analysis

Result: Embeddings Capture Numeracy

Interpolation	List Maximum (5-classes)			Decoding (RMSE)			Addition (RMSE)		
Integer Range	[0,99]	[0,999]	[0,9999]	[0,99]	[0,999]	[0,9999]	[0,99]	[0,999]	[0,9999]
Random Vectors	0.16	0.23	0.21	29.86	292.88	2882.62	42.03	410.33	4389.39
Untrained CNN	0.97	0.87	0.84	2.64	9.67	44.40	1.41	14.43	69.14
Untrained LSTM	0.70	0.66	0.55	7.61	46.5	210.34	5.11	45.69	510.19
Value Embedding	**0.99**	0.88	0.68	**1.20**	11.23	275.50	**0.30**	15.98	654.33
Pre-trained									
Word2Vec	0.90	0.78	0.71	2.34	18.77	333.47	0.75	21.23	210.07
GloVe	0.90	0.78	0.72	2.23	13.77	174.21	0.80	16.51	180.31
ELMo	0.98	0.88	0.76	2.35	13.48	62.20	0.94	15.50	45.71
BERT	0.95	0.62	0.52	3.21	29.00	431.78	4.56	67.81	454.78
Learned									
Char-CNN	0.97	**0.93**	**0.88**	2.50	**4.92**	**11.57**	1.19	**7.75**	**15.09**
Char-LSTM	0.98	0.92	0.76	2.55	8.65	18.33	1.21	15.11	25.37
DROP-trained									
NAQANet	0.91	0.81	0.72	2.99	14.19	62.17	1.11	11.33	90.01
- GloVe	0.88	0.90	0.82	2.87	5.34	35.39	1.45	9.91	60.70

Figure: Interpolation with integers overall result.(Eric et al 2019. Table 4)

Result: Embeddings Capture Numeracy / 2

- Word Vectors Succeed
  - Both word2vec and GloVe significantly outperform the random vector baseline.
  - ELMo are the strongest pre-trained methods

Interpolation	**List Maximum (5-classes)**			**Decoding (RMSE)**			**Addition (RMSE)**		
Integer Range	[0,99]	[0,999]	[0,9999]	[0,99]	[0,999]	[0,9999]	[0,99]	[0,999]	[0,9999]
Random Vectors	0.16	0.23	0.21	29.86	292.88	2882.62	42.03	410.33	4389.39
Pre-trained									
Word2Vec	0.90	0.78	0.71	2.34	18.77	333.47	0.75	21.23	210.07
GloVe	0.90	0.78	0.72	2.23	13.77	174.21	0.80	16.51	180.31
ELMo	0.98	0.88	0.76	2.35	13.48	62.20	0.94	15.50	45.71
BERT	0.95	0.62	0.52	3.21	29.00	431.78	4.56	67.81	454.78

Figure: (Eric et al 2019. Table 4)

Result: Embeddings Capture Numeracy / 3

- Character-level Methods Dominate
  - Char-level CNNs are best architecture for capturing numeracy.
  - Untrained Char-CNN is surprisingly competitive.

Interpolation	**List Maximum (5-classes)**			**Decoding (RMSE)**			**Addition (RMSE)**		
Integer Range	[0,99]	[0,999]	[0,9999]	[0,99]	[0,999]	[0,9999]	[0,99]	[0,999]	[0,9999]
Random Vectors	0.16	0.23	0.21	29.86	292.88	2882.62	42.03	410.33	4389.39
Untrained CNN	0.97	0.87	0.84	2.64	9.67	44.40	1.41	14.43	69.14
Untrained LSTM	0.70	0.66	0.55	7.61	46.5	210.34	5.11	45.69	510.19
ELMo	0.98	0.88	0.76	2.35	13.48	62.20	0.94	15.50	45.71
BERT	0.95	0.62	0.52	3.21	29.00	431.78	4.56	67.81	454.78
Learned									
Char-CNN	0.97	**0.93**	**0.88**	2.50	**4.92**	**11.57**	1.19	**7.75**	**15.09**
Char-LSTM	0.98	0.92	0.76	2.55	8.65	18.33	1.21	15.11	25.37
DROP-trained									
NAQANet	0.91	0.81	0.72	2.99	14.19	62.17	1.11	11.33	90.01
- GloVe	0.88	0.90	0.82	2.87	5.34	35.39	1.45	9.91	60.70

Figure: (Eric et al 2019. Table 4)

Result: Embeddings Capture Numeracy / 4

- Small VS. Large range
  - BERT struggles for large ranges.
  - Linear model is competitive in small ranges on decoding task.
  - Linear model's performance degrades in larger range, especially for BERT.
  - The Value Embedding method fails for large ranges.

Interpolation	**List Maximum (5-classes)**			**Decoding (RMSE)**			**Addition (RMSE)**		
Integer Range	[0,99]	[0,999]	[0,9999]	[0,99]	[0,999]	[0,9999]	[0,99]	[0,999]	[0,9999]
Random Vectors	0.16	0.23	0.21	29.86	292.88	2882.62	42.03	410.33	4389.39
Value Embedding	**0.99**	0.88	0.68	**1.20**	11.23	275.50	**0.30**	15.98	654.33
Pre-trained									
Word2Vec	0.90	0.78	0.71	2.34	18.77	333.47	0.75	21.23	210.07
GloVe	0.90	0.78	0.72	2.23	13.77	174.21	0.80	16.51	180.31
ELMo	0.98	0.88	0.76	2.35	13.48	62.20	0.94	15.50	45.71
BERT	0.95	0.62	0.52	3.21	29.00	431.78	4.56	67.81	454.78

Figure: (Eric et al 2019. Table 4)

Result: Embeddings Capture Numeracy / 5

- Interpolation with floats and negatives
  - Pre-trained and learned embeddings exhibit natural numeracy.
  - BERT struggles on floats, especially on large range.

Interpolation	**List Maximum (5-classes)**	
Float Range	[0.0,99.9]	[0.0,999.9]
Rand. Vectors	0.18 ± 0.03	0.21 ± 0.04
ELMo	0.91 ± 0.03	0.59 ± 0.01
BERT	0.82 ± 0.05	0.51 ± 0.04
Char-CNN	0.87 ± 0.04	0.75 ± 0.03
Char-LSTM	0.81 ± 0.05	0.69 ± 0.02

Figure: Float.[2]

Interpolation	**List Maximum (5-classes)**
Integer Range	[-50,50]
Rand. Vectors	0.23 ± 0.12
Word2Vec	0.89 ± 0.02
GloVe	0.89 ± 0.03
ELMo	0.96 ± 0.01
BERT	0.94 ± 0.02
Char-CNN	0.95 ± 0.07
Char-LSTM	0.97 ± 0.02

Figure: Negative.[2]

Result: Probing Models Struggle to Extrapolate

- Extrapolation problem

  Struggle to extrapolate to values outside training range.

- Extrapolation evaluation

  Train models on a specific integer range and test them on values greater than the largest training number and smaller than the smallest training number.

Result: Extrapolation on Decoding

- predictions using BERT are usually near highest training value
- word2vec prediction decrease monotonically as input increases.

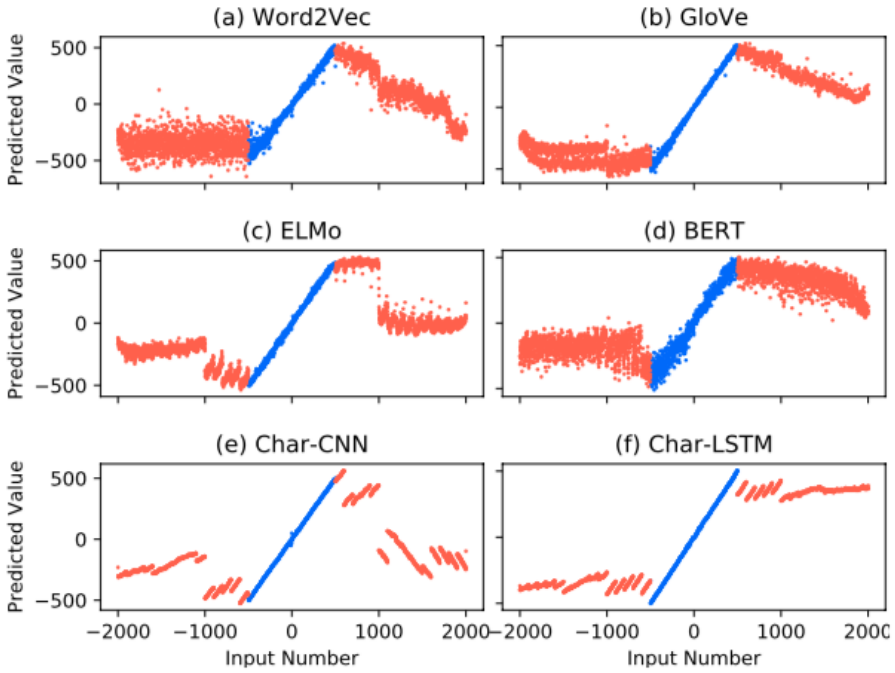

Figure: (Eric et al 2019. Figure 1)

Result: Extrapolation on List Maximum

- Accuracies are closer to those in the interpolation setting
- The probing model struggles to extrapolate when trained on the pre-trained embeddings, especially token vectors.

Extrapolation *Test Range*	**List Maximum (5-classes)** [151,160]	[151,180]	[151,200]
Rand. Vectors	0.17	0.22	0.15
Untrained CNN	0.80	0.47	0.41
Pre-trained			
Word2Vec	0.14	0.16	0.11
GloVe	0.19	0.17	0.21
ELMo	0.65	0.57	0.38
BERT	0.35	0.11	0.14
Learned			
Char-CNN	0.81	0.75	0.73
Char-LSTM	**0.88**	**0.84**	**0.82**
DROP			
NAQANet	0.31	0.29	0.25
- GloVe	0.58	0.53	0.48

Figure: (Eric et al 2019. Table 7)

Conclusion

- Pre-trained representations can naturally encode numeracy.
- ELMo is the best pre-trained methods for numeracy.
- BERT is less exact than ELMo which uses sub-word.
- Char-level better than Token-level $\rightarrow$ better than Sub-token.
- Probing tasks enhance numeracy capability on embeddings.
- Range can quite effect performance of numeracy embedding..
- .....

My Opinions

- Advantages
  - Interesting inspiration and novelty design for probing task.
  - Easy understandable and well structured paper.
  - Source code available online, can be re-implemented.
  - Opportunities for more interesting and useful testings.
- Disadvantages
  - General summarized paper without detailed model setting.
  - Not much explanation why numeracy appears in embedding.

Questions

Reference

[1] Eric et al. (2019) Do NLP Models Know Numbers? Probing Numeracy in Embeddings.

[2] Eric et al. (2019) Do NLP Models Know Numbers? Probing Numeracy in Embeddings. Table 5 & 6.